

Q1. Multiple Threads Using a Single Class

Score: 10.00 TC: 3/3 passed

CODE

```
public class Main {
    public static void main(String[] args) {
        int inputNumber = 153;
        NumberThread t1 = new NumberThread("Prime", inputNumber);
        NumberThread t2 = new NumberThread("Armstrong", inputNumber);
        NumberThread t3 = new NumberThread("Reverse", inputNumber);
        try {
            t1.start();
            t1.join();
            t2.start();
            t2.join();
            t3.start();
            t3.join();
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}

class NumberThread extends Thread {
    int number;
    NumberThread(String name, int number) {
        super(name);
        this.number = number;
    }
    static void checkPrime(int n) {
        boolean isPrime = true;
        if (n < 2) isPrime = false;
        else {
            for (int i = 2; i <= Math.sqrt(n); i++) {
                if (n % i == 0) {
                    isPrime = false;
                    break;
                }
            }
        }
        if (isPrime) System.out.println("Prime Thread: " + n + " is a prime number");
        else System.out.println("Prime Thread: " + n + " is not a prime number");
    }
    static void checkArmstrong(int n) {
        int temp = n;
        int digits = String.valueOf(n).length();
        int sum = 0;
        while (temp != 0) {
            int digit = temp % 10;
            sum += Math.pow(digit, digits);
            temp /= 10;
        }
        if (sum == n) System.out.println("Armstrong Thread: " + n + " is an Armstrong number");
        else System.out.println("Armstrong Thread: " + n + " is not an Armstrong number");
    }
    static void checkReverse(int n) {
        int temp = n;
        int reversed = 0;
        while (temp != 0) {
            int digit = temp % 10;
            reversed = reversed * 10 + digit;
            temp /= 10;
        }
    }
}
```

```
    }  
    System.out.println("Reverse Thread: Reverse of " + n + " = " + reversed);  
}
```

SIMATS Engineering • Department of CSE • CSA0925 — Programming In java • 14-08-2026 • Category: Java Multithreading- Practice -CO3 • Student Submissions Report

```
@Override  
public void run() {  
    String name = getName();  
    if (name.equals("Prime")) {  
        checkPrime(number);  
    } else if (name.equals("Armstrong")) {  
        checkArmstrong(number);  
    } else if (name.equals("Reverse")) {  
        checkReverse(number);  
    }  
}
```

OUTPUT

```
Prime Thread: 153 is not a prime number  
Armstrong Thread: 153 is an Armstrong number  
Reverse Thread: Reverse of 153 = 351
```

Q2. Multiple thread classes

Score: 20.00 TC: 3/3 passed

CODE

```
public class Main {
    public static void main(String[] args) throws InterruptedException {
        int number = 7;
        PrimeThread t1 = new PrimeThread(number);
        FactorialThread t2 = new FactorialThread(number);
        ReverseThread t3 = new ReverseThread(number);

        t1.start();
        t1.join();
        t2.start();
        t2.join();
        t3.start();
        t3.join();
    }
}

class PrimeThread extends Thread {
    int number;
    PrimeThread(int number) {
        this.number = number;
    }
    public void run() {
        boolean isPrime = true;
        if (number < 2) {
            isPrime = false;
        } else {
            for (int i = 2; i <= Math.sqrt(number); i++) {
                if (number % i == 0) {
                    isPrime = false;
                    break;
                }
            }
        }
        if (isPrime) {
            System.out.println("Prime: " + number + " is a prime number");
        } else {
            System.out.println("Prime: " + number + " is not a prime number");
        }
    }
}

class FactorialThread extends Thread {
    int number;
    FactorialThread(int number) {
        this.number = number;
    }
    public void run() {
        long factorial = 1;
        for (int i = 1; i <= number; i++) {
            factorial *= i;
        }
        System.out.println("Factorial: " + number + "! = " + factorial);
    }
}

class ReverseThread extends Thread {
    int number;
    ReverseThread(int number) {
        this.number = number;
    }
    public void run() {
```

```
}  
class ReverseThread extends Thread {  
    int number;  
    ReverseThread(int number) {  
        this.number = number;  
    }  
    public void run() {  
        int temp = number;  
        int reversed = 0;
```

SIMATS Engineering • Department of CSE • CSA0925 — Programming In java • 14-08-2026 • Category: Java Multithreading- Practice -CO3 • Student Submissions Report

```
        while (temp != 0) {  
            int digit = temp % 10;  
            reversed = reversed * 10 + digit;  
            temp /= 10;  
        }  
        System.out.println("Reverse: Reverse of " + number + " = " + reversed);  
    }  
}
```

OUTPUT

```
Prime: 7 is a prime number  
Factorial: 7! = 5040  
Reverse: Reverse of 7 = 7
```

Q3. Thread Synchronization

Score: 10.00 TC: 3/3 passed

CODE

```
public class Main {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();
        Thread threadA = new Thread(new IncrementTask(counter), "ThreadA");
        Thread threadB = new Thread(new IncrementTask(counter), "ThreadB");
        threadA.start();
        threadA.join();
        System.out.println("ThreadA completed");
        threadB.start();
        threadB.join();
        System.out.println("ThreadB completed");
        System.out.println("Final Count = " + counter.count);
    }
}

class Counter {
    int count = 0;
    synchronized void increment() {
        count++;
    }
}

class IncrementTask implements Runnable {
    Counter counter;
    IncrementTask(Counter counter) {
        this.counter = counter;
    }
    public void run() {
        for (int i = 0; i < 1000; i++) {
            counter.increment();
        }
    }
}
```

OUTPUT

```
ThreadA completed
ThreadB completed
Final Count = 2000
```